

CODE EVALUATION - Books

id	Title	Author	Subject	Publisher	Edition	Quantity (in stock)	Price(per book)
1	Data Structure	Lipschute	DS	McGraw	First Edition	4	217.00
2	DOS Guide	NORTRON	OS	PHI	First Edition	3	175.00
3	Turbo C++	Robert Lafore	Prog	Galgotia	Second Edition	5	270.00
4	Dbase Dummies	Palmer	DBMS	PustakM	First Edition	7	130.00
5	Mastering Windows	Cowart	OS	BPB	Third Edition	1	225.00
6	Computer Studies	French	FND	Galgotia	Second Edition	2	75.00
7	COBOL	Stern	Prog	John W	Third Edition	4	1000.00
8	Guide Network	Freed	NET	Zpress	Second Edition	3	200.00
9	Basic for Beginners	Norton	Prog	BPB	First Edition	3	40.00
10	Advanced Pascal	Schildt	Prog	MCGraw	Third Edition	4	350.00

Purchase

order_no	Book_id	item_quantity	purchase_amt	date	customer_id	salesman_id
70001	3	3	810.00	2022-11-05	3005	5002
70009	4	4	520.00	2022-11-10	3001	5005
70002	5	1	225.00	2022-11-09	3002	5001
70004	6	2	150.00	2022-11-08	3009	5003
70007	1	2	434.00	2022-11-08	3005	5002
70008	8	1	200.00	2022-11-01	3007	5001
70010	7	2	2000.00	2022-11-03	3004	5006
70003	10	2	700.00	2022-11-04	3009	5003
70012	9	2	80.00	2022-11-02	3008	5002
70011	2	2	350.00	2022-11-07	3003	5007
70013	7	2	2000.00	2022-11-06	3002	5001

Customer

id	FirstName	LastName	Street	City	State	Phone	Email
3001	Ben	Miller	101,Candy Rd	Redmond	WA	9845367123	ben@gmail.com
3002	Garrett	Vargas	102, Acorn Avenue	Calgary	AB	9789011111	garrett@gmail.com
3003	Gabe	Mares	1061, Buskrit Avenue	Edmonds	WA	9944433222	gabe@gmail.com
3004	Reuben	Dsa	1064, Slow Creek Road	Seattle	WA	9988844555	reuben@gmail.com
3005	Gordon	Hee	108, LakeSide Court	Bellevue	WA	8855577234	gordon@gmail.com
3006	Karan	Khanna	1102, Ravenwood	Seattle	WA	7788966445	karan@gmail.com
3007	Francois	Ajenstat	1144, Paradise Ct	Issaquah	WA	7889900123	francois@gmail.com
3008	Sariya	Hampa	1185, Dallas Drive	Everett	WA	9667785544	sariya@gmail.com
3009	Krik	Koenigs	1220, Bradford Way	Seattle	WA	9774455443	krik@gmail.com
3010	Kim	Ralls	1226, Shoe St	Bothell	WA	8770033221	kim@gmail.com

Salesman

id	Name	City	Commission
5001	James Hoog	New York	0.15
5002	Nail Knite	Paris	0.13
5003	Lauson Hen	Germany	0.12
5005	Pit Alex	London	0.11
5006	Mc Lyon	Paris	0.14
5007	Paul Adam	Rome	0.13

Evaluation: 1

1. Insert all documents to Books and Purchase collections
2. create a text search for Title

```
> db.books.createIndex({title:"text"})  
< title_text  
> db.books.find({$text:{$search:"COBOL"}})
```

3. select Title, sum(price) from Books order by Title ASC

```
db.books.aggregate([  
  {$group: {_id:"$title",TotalPrice:{$sum:"$price"}}},  
  {$project: {title:"$_id",_id:0,TotalPrice:1}},  
  {$sort: {title:1}}  
)
```

4. select Book_id,item_quantity from Purchase Group by Book_id
Having count(item_quantity)>1

```
> db.purchase.aggregate([  
  {$group: {_id:"$book_id",item_quantity:{$sum:"$item_quantity"},count:{$sum:1}}},  
  {$match: {count:{$gt:1}}},  
  {$project: {_id:0,"bookId": "$_id",item_quantity:1}}  
)
```

5. Query to execute the sum of the price group by Edition

```
> db.books.aggregate([  
  {$group: {_id:"$edition",totalPrice:{$sum:"$price"}}},  
  {$project: {_id:0,"edition": "$_id",totalPrice:1}}  
)
```

6. select Title as _id, sum(quantity*price) as totalAmount from Books
order by totalAmount in ASC

```
> db.books.aggregate([  
  {$group: {_id:"$title",totalAmount:{$sum:{$multiply: ["$quantity","$price"]}}},  
  {$sort: {totalAmount:1}}  
)
```

7. Query to print only the average of purchase_amt

```
> db.purchase.aggregate([  
  {$group: {_id:null,avgPurchaseAmt:{$avg:"$purchase_amt"}}}  
)
```

8. select Title as _id, sum(quantity*price) as totalAmount from Books where totalAmount>=1000.00 order by totalAmount in desc

```
> db.books.aggregate([
  {$group: {_id: "$title", totalAmount: {$sum: {$multiply: ["$quantity", "$price"]}}}},
  {$match: {totalAmount: {$gte: 1000}}},
  {$sort: {totalAmount: -1}}
])
```

9. update the Book name COBOL as COBOL_new_Edition using Variable in let

```
> let bookName = "COBOL"
let newName = "COBOL_new_Edition"
db.books.updateOne(
  {title: bookName},
  {$set: {title: newName}}
)
```

10. Query to set unique document in Books Collection

```
> db.books.createIndex({ id: 1 }, { unique: true })
< id_1
> db.books.getIndexes()
< [
  { v: 2, key: { _id: 1 }, name: '_id_',
    {
      v: 2,
      key: { _fts: 'text', _ftsx: 1 },
      name: 'title_text',
      weights: { title: 1 },
      default_language: 'english',
      language_override: 'language',
      textIndexVersion: 3
    }
  },
  { v: 2, key: { id: 1 }, name: 'id_1', unique: true }
]
```

11. Autoincrement in MongoDB to store sequence of Unique id for Books collection

```
> db.books.insertOne({
  id: db.books.find().count() + 1,
  title: "Python Programming",
  author: "Guido Van Rossum",
  subject: "Prog",
  publisher: "McGraw",
  edition: "First Edition",
  quantity: 3,
  price: 450.00
})
```

12. select documents where purchase_amt%4=0

```
> db.purchase.aggregate([
  {$match:{$expr:{$eq:[$mod:["$purchase_amt",4]],0}}}
])
```

Evaluation: 2

1. Query to join the two collections of Customer and Salesman

```
> db.purchase.aggregate([
  {$lookup:{from:"customer",localField:"customer_id",
    foreignField:"id",as:"customerDetails"}},
  {$lookup:{from:"salesman",localField:"salesman_id",
    foreignField:"id",as:"salesmanDetails"}},
  {$unwind:"$customerDetails"},
  {$unwind:"$salesmanDetails"},
  {$project:{_id:0,customerName:{$concat:["$customerDetails.firstName",
    " ", "$customerDetails.lastName"] },
    salesmanName: "$salesmanDetails.name", purchase_amt: 1}}
])
```

2. select FirstName, LastName, City, State from Customer where State='WA' order by City

```
> db.customer.aggregate([
  {$match:{state:"WA"}},
  {$sort:{city:1}},
  {$project:{_id:0,firstName:1,lastName:1,city:1,state:1}}
])
```

3. update the commission of Salesman id=5005 to 0.13 and display the output immediately after the updation.

```
> db.salesman.findOneAndUpdate({id:5005},{set:{commission:0.13}},{returnDocument:"after"})
```

4. Select the documents where the name starts with 'G'

```
> db.salesman.find({name:{$regex: "^G"}})
```

5. Create a text search for Name in salesman and search the name = 'Pit Alex' and display the output

```
> db.salesman.createIndex({name:"text"})
< name_text
> db.salesman.find({$text:{$search:"Pit Alex"}})
< {
  _id: ObjectId('6a1542ec92892b3bb212d2da'),
  id: 5005,
  name: 'Pit Alex',
  city: 'London',
  commission: 0.13
}
```

6. Select the documents from Customer starting with 'k' using regex option keyword

```
> db.customer.find({firstName:{$regex:"K",$options:"i"}})
```

7. Fetching the last nth document from Customer

```
> const n = 3
const total = db.customer.countDocuments()
db.customer.find().skip(total - n).limit(1)
```

8. pipeline operator to join the two collections based on purchase.salesman_id=salesman.id

```
> db.purchase.aggregate([
  {$lookup:{
    from:"salesman",
    localField:"salesman_id",
    foreignField:"id",
    as:"salesmanDetails" }},
  {$unwind:"$salesmanDetails"},
  {$project:{_id:0,order_no:1,purchase_amt:1,
    salesmanName:"$salesmanDetails.name",
    salesmanCity:"$salesmanDetails.city"
  }}
])
```

9. Joining the two collections into one document output - select customer.FirstName, customer.LastName, customer.Phone, Customer.Email, Purchase.item_quantity, Purchase.purchase_amt from Customer left join on Purchase where Customer.id=Purchase.customer_id order by Customer.firstName

```
> db.customer.aggregate([
  {$lookup:{
    from: "purchase",
    localField: "id",
    foreignField: "customer_id",
    as: "purchaseDetails" }},
  {$unwind:"$purchaseDetails"},
  {$project:{_id:0, firstName:1, lastName:1, phone:1,
    email:1,
    item_quantity: "$purchaseDetails.item_quantity",
    purchase_amt: "$purchaseDetails.purchase_amt"}},
  {$sort:{firstName:1}}
])
```

10.

```
> db.students3.find()
< {
  _id: 1,
  tests: [
    95,
    92,
    90
  ],
  average: 92,
  grade: 'A',
  lastUpdate: 2020-01-23T05:18:40.013Z
}
{
  _id: 2,
  tests: [
    70,
    75,
    82
  ],
  lastUpdate: 2019-01-01T00:00:00.000Z
}
```

Find the Average and grade of `_id:2` and update the ISODate to current date and time.

```
> db.students3.updateOne(
  {_id:2},
  [{ $set:{
    average:{ $avg:"$tests"},
    grade:"B",
    lastUpdate:new Date()
  }}]
)
```

```
> db.students3.find()
< {
  _id: 1,
  tests: [
    95,
    92,
    90
  ],
  average: 92,
  grade: 'A',
  lastUpdate: 2020-01-23T05:18:40.013Z
}
{
  _id: 2,
  tests: [
    70,
    75,
    82
  ],
  lastUpdate: 2026-05-26T14:18:41.706Z,
  average: 75.66666666666667,
  grade: 'B'
}
```